

System Programming Project 3

담당 교수 : 김영재 교수님

이름 : 이석희

학번 : 20200595

1. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.
- (주식 서버를 만드는 전체적인 개요에 대해서 작성하면 됨.)

본 프로젝트의 목표는 다수의 클라이언트가 동시에 접속하여 주식 정보를 조회하거나 주문할 수 있는 **멀티클라이언트 기반 주식 서버**를 구현하는 것이다. 서버는 클라이언트로부터 buy, sell, show, exit 명령을 처리하고, 그에 따라 메모리에 저장된 주식 데이터를 갱신하거나 출력한다. 또한 모든 클라이언트가 연결을 종료한 시점에 주식 정보를 stock.txt 파일에 저장함으로써 데이터의 일관성을 유지하도록 설계하였다. 전체적인 구현은 네트워크 프로그래밍과 동시성 제어에 대한 이해를 바탕으로, 안정적이고 효율적인 서버 구조를 구축하는 데 중점을 두었다.

2. 개발 범위 및 내용

A. 개발 범위

- 아래 항목을 구현했을 때의 결과를 간략히 서술

1. Task 1: Event-driven Approach

- select() 함수를 이용해 하나의 스레드로 여러 클라이언트의 요청을 동시에 처리할 수 있도록 I/O Multiplexing 방식으로 주식 서버를 구현하였다.
- 클라이언트가 명령어(buy, sell, show, exit)를 전송하면 서버는 해당 요청을 처리하고 응답을 보낸다.
- 모든 클라이언트가 연결을 종료한 시점에서만 stock.txt 를 업데이트하도록 구현하여 불필요한 디스크 접근을 방지했다.

2. Task 2: Thread-based Approach

- pthread 를 이용해 서버는 각 클라이언트마다 새로운 스레드를 생성하여 병렬적으로 요청을 처리할 수 있도록 하였다.
- 공유 자원(stock tree, stock.txt, byte_cnt)에 대해서는 뮤텍스 및 세마포어를 이용하여 동시성 문제를 해결하였다.
- 마지막 클라이언트 연결 종료 시점에만 stock.txt 를 저장하는 방식은 동일하게 유지하였다.

3. Task 3: Performance Evaluation

- 여러 환경과 setting을 만들어 그에 따른 throughput을 계산했고, 그 결과를 그래프로 그려 task 1과 task 2를 비교하기 쉽도록 하였다.

B. 개발 내용

- 아래 항목의 내용만 서술
- (기타 내용은 서술하지 않아도 됨. 코드 복사 붙여 넣기 금지)

- Task1 (Event-driven Approach with select())

- ✓ Multi-client 요청에 따른 I/O Multiplexing 설명

select() 함수를 이용해 단일 스레드에서 여러 클라이언트 소켓을 동시에 관리하였다.

- 서버는 listenfd와 함께 연결된 클라이언트 소켓들을 fd_set(read_set)에 등록한다.
- select() 시스템 콜을 통해 읽기 이벤트가 발생한 소켓을 감지하고, 그 결과는 ready_set에 저장된다.
- ready_set에 포함된 파일 디스크립터에 대해 Rio_readlineb()로 명령어를 읽고, buy, sell, show, exit 명령을 처리한다.
- 소켓이 EOF 상태가 되면 해당 소켓은 read_set에서 제거되며 Close() 호출로 연결을 종료한다.

이 방식을 통해, 하나의 서버 프로세스가 수십 개의 클라이언트와 동시 통신을 처리할 수 있게 된다.

- ✓ epoll과의 차이점 서술

현재 구현에서는 select() 시스템 콜을 사용하여 다중 클라이언트의 이벤트를 감지하고 처리하고 있다. select() 방식은 구현이 비교적 간단하고 소규모 서버 환경에서는 안정적으로 동작하지만, 클라이언트 수가 많아질수록 전체 파일 디스크립터 집합을 매번 순회해야 하기 때문에 성능이 급격히 저하되는 단점이 있다. 반면, epoll은 이벤트가 발생한 파일 디스크립터만을 반환하므로 $O(1)$ 에 가까운 성능을 제공하며, 수천 개 이상의 연결도 효율적으로 처리할 수 있다. 또한 epoll은 FD_SETSIZE와 같은 파일 디스크립터 개수 제한이 없어 대규모 서비스에 적합하다. 하지만 구조가 복잡하고 구현 난이도가 높아지는 단점이 있다.

- Task2 (Thread-based Approach with pthread)

✓ Master Thread의 Connection 관리

- 서버는 main() 루프 내에서 클라이언트의 연결 요청을 수신하고, Accept()를 통해 connfd 를 획득한다.
- 연결이 수립되면 connfd 를 힙에 동적으로 할당하고, 이 값을 인자로 하여 새로운 스레드를 Pthread_create()를 통해 생성한다.
- 각 연결마다 독립적인 스레드가 생성되며, 병렬로 명령어를 처리할 수 있게 된다.
- 이 과정에서 client_count 전역 변수를 사용하여 활성화된 클라이언트 수를 추적하고, 마지막 클라이언트가 종료될 때 stock.txt 를 저장하도록 구현하였다.

✓ Worker Thread Pool 관리하는 부분에 대해 서술

- 각 스레드는 thread() 함수 내에서 분리(detach)되어 pthread_detach()로 독립 실행된다. 메모리 해제를 위해 connfd 포인터는 Free()된다.
- handle_request()는 해당 스레드에서 클라이언트로부터 반복적으로 명령어를 읽고 응답을 보낸다.
- 동시성 문제를 해결하기 위해 다음과 같은 동기화 장치를 적용하였다:
 - stock 트리 노드마다 readcnt, mutex, w를 포함하여 **reader-writer lock**을 구현하였다.
 - ◆ show: 다수 reader가 동시에 접근 가능
 - ◆ buy, sell: writer 단독 접근
 - 파일 저장은 save_mutex를 이용해 동기화하여 race condition을 방지하였다.
 - 전체 처리 바이트 수(byte_cnt)를 보호하기 위한 byte_cnt_mutex도 사용되었다.
- 스레드가 종료될 때마다 client_count를 1 감소시키고, 만약 이것이 마지막 클라이언트였다면 save_stock_table()을 호출한다.

- Task3 (Performance Evaluation)

✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

본 성능 평가에서 설정한 핵심 metric은 **throughput (처리량)**이며, 이는 단위 시간 동안 서버가 처리한 총 요청 수로 정의된다. 이 지표를 선택한 이유는, 서버가 여러 클라이언트의 요청을 얼마나 효율적으로 동시에 처리할 수 있는지를 정량적으로 측정할 수 있기 때문이다. 특히, buy, sell, show와 같이 명령의 처리 시간과 동기화 요구 조건이 서로 다

른 작업들이 혼재된 상황에서, 처리량은 구조적 차이에 따른 성능 격차를 비교하기에 적합한 기준이 된다.

throughput 측정은 다양한 부하 조건에서 서버가 얼마나 많은 요청을 처리할 수 있는지를 알아보기 위해 수행된다. 이를 위해 총 처리된 요청 수를 수행에 걸린 총 시간(초)으로 나누어 throughput 값을 계산하며, 클라이언트 수와 명령 수의 변화에 따라 성능이 어떻게 달라지는지를 확인한다.

실험은 세 가지 클라이언트 실행 환경으로 구성되며, 각각 run_experiment.sh 스크립트를 통해 일괄적으로 실행하였다:

1. **Random** – buy, sell, show 명령을 무작위로 수행하는 환경
2. **only buy or sell** – buy 또는 sell 명령만 수행하는 환경
3. **only show** – show 명령만 반복 수행하는 환경

각 환경마다 다음의 네 가지 설정을 기준으로 실험을 수행하였다:

1. MAX_CLIENT 10, ORDER_PER_CLIENT 10
2. MAX_CLIENT 30, ORDER_PER_CLIENT 10
3. MAX_CLIENT 10, ORDER_PER_CLIENT 30
4. MAX_CLIENT 30, ORDER_PER_CLIENT 30

클라이언트 수는 1에서 MAX_CLIENT까지 변화시키며 task1_stockserver를 실행하고, 클라이언트 수를 네 번째 인자로 전달하는 방식으로 서버와 클라이언트를 실행하였다. 서버는 처리된 모든 요청에 대해 응답하고, 클라이언트 실행 종료 시 전체 수행 시간과 처리된 요청 수를 기반으로 throughput을 계산하였다. 측정된 결과는 CSV 파일로 저장되어 이후 그래프 형태로 시각화되며, 서버 구조별 성능 차이를 비교하는 데 활용된다.

✓ Configuration 변화에 따른 예상 결과 서술

- **Random 환경 (buy, sell, show 무작위 혼합)**

- **클라이언트 수 증가**

- *I/O Multiplexing*: select()는 모든 파일 디스크립터를 순회하므로, 클라이언트 수가 많아질수록 처리 지연이 발생할 가능성이 높음.
- *Thread*: 병렬 처리가 가능하므로 throughput 이 더 잘 유지될 것으로 예상되지만, 중간 수준의 클라이언트 수에서 일시적 context switch 나 락 충돌로 성능이 급락할 수 있음.

- 요청 수 증가 (ORDER_PER_CLIENT)

- *I/O Multiplexing*: 요청이 많을수록 순차 처리의 병목이 두드러짐. 처리량 증가에 한계가 있을 것으로 보임.
- *Thread*: 각 스레드가 자체적으로 여러 요청을 처리하므로 효율적. 요청 수가 많을수록 병렬 처리의 이점이 커질 것으로 예상됨.

- Buy/Sell 환경 (쓰기 연산 중심)

- 클라이언트 수 증가

- *I/O Multiplexing*: 순차적으로 write 처리하므로 클라이언트 수 증가 시 처리 지연이 점점 커짐.
- *Thread*: 클라이언트 수가 많아질수록 write lock 경합이 심해져 성능이 급격히 저하될 가능성 있음.

- 요청 수 증가

- *I/O Multiplexing*: 명령 수가 늘어날수록 직렬 처리 병목이 명확히 나타남.
- *Thread*: 요청 수가 많고 write 연산이 집중되면 각 스레드 간 lock 충돌이 누적되어 throughput 이 불안정해질 수 있음.

- Show-only 환경 (읽기 연산 중심)

- 클라이언트 수 증가

- *I/O Multiplexing*: 많은 수의 클라이언트 연결을 순회하는 데 드는 비용 증가로 인해, 특정 시점 이후 throughput 하락이 발생할 수 있음.
- *Thread*: reader lock 을 통해 병렬 read 가 가능하므로, 클라이언트 수가 많아져도 매우 높은 처리량을 안정적으로 유지할 것으로 기대됨.

- 요청 수 증가

- *I/O Multiplexing*: read 는 경량 작업이므로 요청 수 증가에도 일정 수준의 처리량은 유지되지만, 루프 병목이 누적될 수 있음.

- *Thread*: read 작업은 거의 락 경쟁이 없기 때문에 요청 수가 많아질수록 병렬 처리 효과가 더 크게 나타날 것으로 예상됨.

C. 개발 방법

- B.의 개발 내용을 구현하기 위해 어느 소스코드에 어떤 요소를 추가 또는 수정할 것인지 설명. (함수, 구조체 등의 구현이나 수정을 서술)

기존의 stockserver.c는 echo() 함수를 기반으로 한 **반복적(iterative) 처리** 구조였다. 이를 Task 1과 Task 2의 요구 사항에 맞춰 다음과 같이 기능을 추가 및 수정하였다.

두 server 모두 주식에 대한 정보가 필요하므로 stock 구조체를 구현해 binary tree의 node에 item으로 저장하였다. 이 때, I/O multiplexing와 달리 thread를 기반의 server는 주식 정보의 동기화를 위한 readcnt와 read 전용 mutex, write 전용 mutex가 stock 구조체에 추가되었다.

Task1의 stock 구조체

```
typedef struct { /* Represents a stock info */
    int ID;          /* Stock ID */
    int left_stock;  /* The amount of left stock */
    int price;       /* Price of stock */
} stock;
```

Task2의 stock 구조체

```
typedef struct { /* Represents a stock info */
    int ID;          /* Stock ID */
    int left_stock;  /* The amount of left stock */
    int price;       /* Price of stock */
    int readcnt;     /* Reference count (reader count) */
    sem_t mutex;     /* Semaphore to protect readcnt */
    sem_t w;         /* Semaphore to serialize writers (and first/last reader) */
} stock;
```

Task 1: I/O Multiplexing 기반 다중 클라이언트 처리

전체 흐름 요약

1. 서버는 listenfd를 열고, pool 구조체를 초기화하여 select()를 사용할 준비를 한다.
2. select()를 통해 클라이언트 연결 요청 및 읽기 이벤트를 감지한다.
3. 새로운 클라이언트가 연결되면 pool에 등록하고, 기존 클라이언트의 요청은 순차적으로 처리한다.
4. 클라이언트로부터 buy, sell, show, exit 명령을 수신하여 그에 맞게 주식 데이터를 수정하거나 조회한다.
5. exit 명령 또는 EOF가 발생하면 해당 클라이언트를 연결 목록에서 제거하고 종료 처리한다.
6. 모든 클라이언트가 연결을 종료한 시점에서만 stock.txt 파일을 저장한다.
7. 서버는 종료되지 않고 새로운 연결을 계속해서 수락하며 대기 상태를 유지한다.

상세 구현

1. pool 구조체 추가

기존의 구조를 I/O multiplexing 방식으로 변경하기 위해 pool이라는 구조체를 추가하여, 여러 클라이언트의 파일 디스크립터를 fd_set으로 효율적으로 관리할 수 있도록 구성하였다. 이 구조체에는 최대 디스크립터 번호, 현재 읽기 준비된 디스크립터 수, 클라이언트별 파일 디스크립터 배열 및 버퍼를 포함하였다.

```
/* My structure in task_1 */
typedef struct { /* Represents a pool of connected descriptors */
    int maxfd; /* Largest descriptor in read_set */
    fd_set read_set; /* Set of all active descriptors */
    fd_set ready_set; /* Subset of descriptors ready for reading */
    int nready; /* Number of ready descriptors from select() */
    int maxi; /* High water index into clientfd[] array */
    int clientfd[FD_SETSIZE]; /* Set of active descriptors */
    rio_t clientrio[FD_SETSIZE]; /* Set of active read buffers */
} pool;
```

2. init_pool(), add_client(), handle_request() 함수 구현

init_pool(), add_client(), handle_request()와 같은 새로운 함수를 정의하였다. init_pool()은 listenfd를 포함하여 초기 상태를 설정하며, add_client()는 새로 접속한 클라이언트를

pool에 추가한다. `handle_request()`는 `select()`를 통해 준비된 디스크립터를 순회하며 클라이언트 요청을 처리한다.

3. 클라이언트 수가 0이 될 때 stock 저장

- `exit` 명령어 또는 EOF 수신 시 모든 연결이 끊겼는지 검사하고, 모든 연결이 종료되었을 때 `save_stock_table()`을 호출해 `stock.txt`를 update한다.

Task 2: Thread 기반 다중 클라이언트 처리

전체 흐름 요약

1. 서버는 `listenfd`를 열고 클라이언트 연결 요청을 수신한다.
2. 클라이언트가 연결될 때마다 새로운 스레드를 `pthread_create()`로 생성한다.
3. 각 스레드는 `buy`, `sell`, `show`, `exit` 명령을 처리하며,
 - i. `buy`, `sell`은 writer lock을,
 - ii. `show`는 reader lock을 통해 동기화를 수행한다.
4. 스레드는 명령 처리 후 연결을 종료하고, 전역 변수 `client_count`를 감소시킨다.
5. `client_count`가 0이 되는 순간, 즉 마지막 클라이언트가 종료되었을 때 `stock.txt` 파일을 저장한다.
6. 메인 스레드는 계속해서 클라이언트 연결을 기다리며 서버는 종료되지 않고 운영을 지속한다.

상세 구현

1. 스레드 생성 및 종료 관리

스레드 기반 구현에서는 `pthread` 라이브러리를 활용하여 클라이언트가 접속할 때마다 새로운 스레드를 생성하도록 변경하였다. 메인 스레드는 클라이언트의 연결을 수락한 뒤, 각 연결에 대해 `pthread_create()`로 독립적인 스레드를 생성하며, 이 스레드는 `handle_request()` 함수를 통해 클라이언트의 명령을 처리한다. Task 2에서도 Task 1과 마찬가지로 `handle_request()` 함수에서 `handle_buy()`, `handle_sell()`, `handle_show()` 함수들을 호출해 클라이언트의 요청을 처리한다.

- 기존 서버 루프에 다음 추가:

```
connfdp = Malloc(sizeof(int));
*connfdp = Accept(listenfd, (SA *)&clientaddr, &clientlen);
Pthread_create(&tid, NULL, thread, connfdp);
```

- thread() 함수는 클라이언트 하나당 독립적으로 요청을 처리하며 종료 시 client_count를 감소시킨다.

2. 스레드 동기화 변수 추가

공유 자원에 대한 동기화가 필요하므로 다양한 mutex와 세마포어를 도입하였다. 서버의 전체 바이트 수를 기록하는 byte_cnt는 byte_cnt_mutex로 보호되고, 현재 접속 중인 클라이언트 수를 관리하는 client_count는 count_mutex로 보호된다. 또, stock.txt 파일에 주식 정보를 update하는 권한은 save_mutex로 보호된다.

- client_count, byte_cnt 등 공유 변수 보호와 stock.txt update를 위한 mutex 사용:

```
pthread_mutex_t save_mutex = PTHREAD_MUTEX_INITIALIZER; // stock.txt을 보호

int byte_cnt = 0; // Modified: 총 바이트 수
pthread_mutex_t byte_cnt_mutex = PTHREAD_MUTEX_INITIALIZER; // Modified: byte_cnt 보호

/* Modified: 전역 클라이언트 카운터와 그 보호용 뮤텁스 추가 */
int client_count = 0; // Modified: 활성 클라이언트 수
pthread_mutex_t count_mutex = PTHREAD_MUTEX_INITIALIZER; // Modified: client_count 보호
```

3. 읽기/쓰기 명령에 따라 lock 적용

앞서 언급했듯이 stock 구조체에 readcnt, mutex, w 필드를 추가하여 reader-writer lock을 구현하였다. handle_show() 함수에서는 여러 스레드가 동시에 데이터를 읽을 수 있도록 하되, readcnt를 이용해 첫 번째 reader가 들어올 때 write를 차단하고, 마지막 reader가 나갈 때 write를 다시 허용한다. handle_buy()와 handle_sell()은 writer 전용으로 처리되어 데이터 무결성을 유지한다.

- handle_show(): 다중 reader 허용 (readcnt, mutex, w 활용)
- handle_buy()/handle_sell(): writer만 접근 가능하도록 P(&w) 사용

Task 3: Performance Evaluation

Task 3에서는 서버의 성능을 다양한 조건에서 자동으로 측정하고 시각화할 수 있도록 별도의 클라이언트 실행 프로그램들과 자동화 스크립트를 설계 및 구현하였다.

먼저, 실험 환경별로 명령 유형을 제어할 수 있도록 세 가지 버전의 클라이언트 프로그램을 작성하였다. multclient1.c는 buy, sell, show 명령을 무작위로 발생시키는 일반 환경, multclient2.c는 buy 또는 sell 명령만 수행하는 환경, 그리고 multclient3.c는 show 명령

만 반복적으로 요청하는 환경을 구성한다. 각 프로그램은 `fork()`를 통해 다수의 클라이언트를 병렬로 생성하며, `ORDER_PER_CLIENT` 매크로를 통해 명령 횟수를 제어할 수 있도록 구성하였다. 클라이언트는 명령을 전송하고 응답을 수신한 뒤 종료되며, 모든 자식 프로세스는 `waitpid()`를 통해 종료를 기다린다.

이러한 세 개의 클라이언트 프로그램을 자동으로 실행하고 측정 결과를 수집하기 위해, `run_experiment.sh` 스크립트를 작성하였다. 이 스크립트는 `multiclient1`, `multiclient2`, `multiclient3`를 순차적으로 실행하며, 각 실험에 대해 클라이언트 수를 1에서 최대 설정 수(`MAX_CLIENT`)까지 증가시키면서 실행 시간을 측정한다. 측정은 `date +%s.%N`으로 시작과 종료 시점을 기록하고, 클라이언트 수와 요청 수로부터 `throughput`(초당 처리 요청 수)을 계산하여 CSV 파일(`multiclient1_result.csv` 등)에 저장한다. 이 스크립트는 각 실험 환경을 자동화하고 반복 실행 가능하게 해주며, 서버의 구조나 명령 패턴에 따라 성능 차이를 수치화하는 기반을 마련한다.

마지막으로, 수집된 성능 결과를 시각화하기 위해 `plot_comparison.py` 파일을 작성하였다. 이 파이썬 스크립트는 각 실험 환경에서의 결과를 읽어들이고, I/O 기반 서버와 Thread 기반 서버의 `throughput`을 한 그래프에 비교하여 출력한다. 클라이언트 수를 x축으로, 처리량을 y축으로 하여 그래프를 생성하며, `comparison_mode1.png`와 같이 자동 저장되도록 하였다.

3. 구현 결과

- 2번의 구현 결과를 간략하게 작성

- 미처 구현하지 못한 부분에 대해선 디자인에 대한 내용도 추가

- Task 1에서는 `select()` 기반의 I/O Multiplexing 방식을 통해 하나의 서버 프로세스가 다수의 클라이언트 요청을 동시에 처리할 수 있도록 구현하였다. pool 구조체를 이용하여 클라이언트 소켓들을 효과적으로 관리하였고, 클라이언트 연결 해제 시 모든 연결이 종료되었는지를 확인하여 `stock.txt`를 저장하는 방식으로 디스크 접근을 최소화하였다.
- Task 2에서는 클라이언트마다 독립적인 스레드를 생성하는 방식으로 병렬 처리를 구현하였다. 각 스레드는 `pthread`를 통해 생성되며, `stock` 정보를 보호하기 위해 `reader-writer lock`을 구현하였다. 또한 공유 자원에 대해 `mutex`를 활용하여 동시성 문제를 해결하였다. 마지막 클라이언트 종료 시에만 주식 정보를 저장함으로써 Task 1과 동일한 저장 조건을 유지하였다.
- 두 방식 모두 `buy`, `sell`, `show`, `exit` 명령을 정상적으로 처리하였으며, 클라이언트 수가 많아지는 환경에서도 서버가 안정적으로 동작함을 확인하였다.
- 조금 아쉬웠던 점은, 두 server 모두 `'Rio_writen(connfd, response, MAXLINE)'` 코드로 server의 응답을 클라이언트에게 전달하는데, 전달하는 응답에 비해 `MAXLINE`의 크기가

커 응답에 많은 대역폭이 쓰이는 점이였다. 이를 해결하려면 클라이언트쪽에서 Rio_readlineb() 함수로 응답을 처리하면 됐는데, 이 프로젝트에서는 multclient.c 를 수정할 수 없었기 때문에 Rio_writen() 함수로 응답을 전달하였다.

4. 성능 평가 결과 (Task 3)

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

Run_experiment.sh 파일에서는 클라이언트 실행 직전에 시작 시간을 기록하고, 모든 클라이언트가 종료된 직후에 종료 시간을 기록하여 그 사이의 전체 실행 시간 (elapsed)을 계산하도록 되어있다.

```
start=$(date +%s.%N)
./multclient${mode} $HOST $PORT $clients > /dev/null
end=$(date +%s.%N)
elapsed=$(echo "$end - $start" | bc)
```

해당 결과는 아래와 같이 csv 파일로 저장되도록 구현하였기에 터미널에 별도의 문구가 출력되지 않았다.

예시: I/O multiplexing으로 Random 환경에서 MAX_CLIENT 10, ORDER_PER_CLIENT 10로 실험한 결과

clients	order_per	time_sec	throughput
1	10	1.013381	9.8679
2	10	1.01207	19.7614
3	10	1.013344	29.6049
4	10	1.014977	39.4097
5	10	1.014691	49.276
6	10	1.016529	59.0243
7	10	1.017779	68.7772
8	10	1.017055	78.6584
9	10	1.018874	88.3328
10	10	1.020468	97.9942

앞서 설명했듯이 세 가지 환경에서 서로 다른 setting 네 가지로 분류하여 실험을 진행하였다. 환경과 setting은 아래와 같다.

세 가지 환경

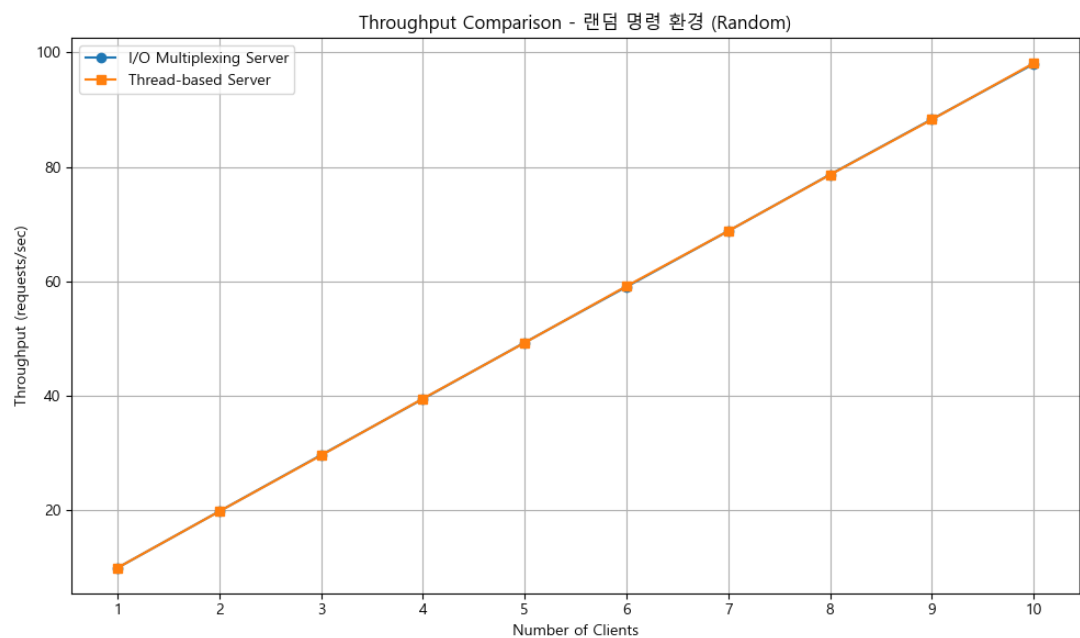
1. **Random** – buy, sell, show 명령을 무작위로 수행하는 환경
2. **only buy or sell** – buy 또는 sell 명령만 수행하는 환경
3. **only show** – show 명령만 반복 수행하는 환경

네 가지 setting

1. MAX_CLIENT 10, ORDER_PER_CLIENT 10
2. MAX_CLIENT 30, ORDER_PER_CLIENT 10
3. MAX_CLIENT 10, ORDER_PER_CLIENT 30
4. MAX_CLIENT 30, ORDER_PER_CLIENT 30

1. [환경 1] Random – Setting 1 결과 분석

(MAX_CLIENT = 10, ORDER_PER_CLIENT = 10)

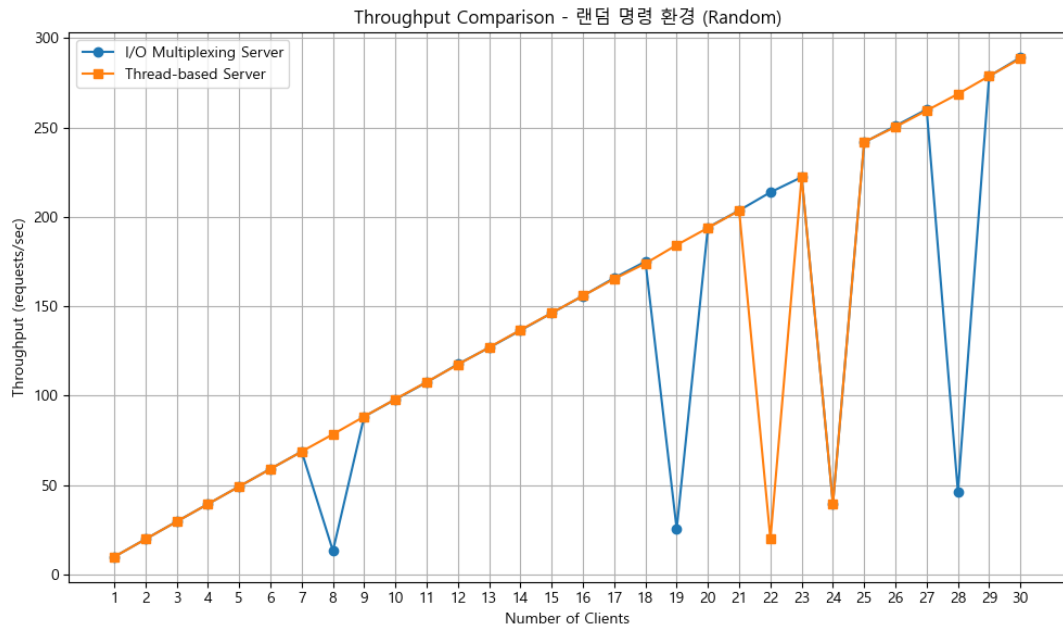


클라이언트 수를 1에서 10까지 증가시키며 랜덤 명령을 수행한 결과, **I/O Multiplexing 서버와 Thread 기반 서버 모두 거의 동일한 throughput 증가 추세**를 보였다. 처리량은 클라이언트 수에 비례하여 선형적으로 증가하였고, 구조적 병목이나 락 경합은 관찰되지 않았다.

이는 낮은 부하 조건에서는 두 방식 모두 안정적으로 작동하며, 구조적 성능 차이가 거의 없음을 의미한다.

2. [환경 1] Random – Setting 2 결과 분석

(MAX_CLIENT = 30, ORDER_PER_CLIENT = 10)



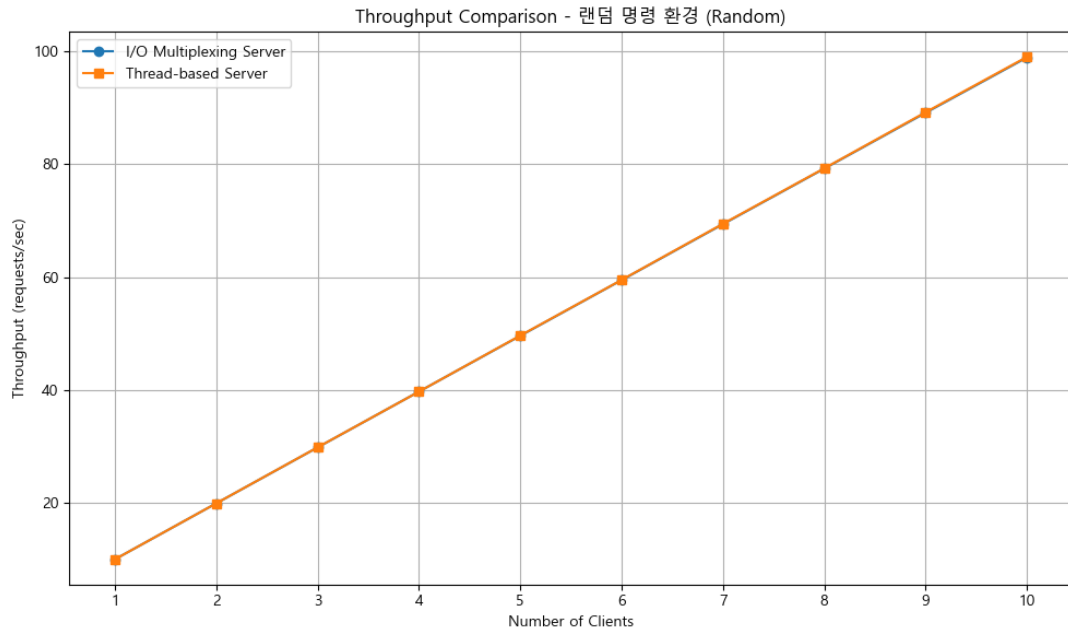
클라이언트 수가 많아진 이번 설정에서, 그래프는 **Thread 기반 서버가 I/O Multiplexing 방식보다 높은 throughput을 유지함**을 보여준다. 클라이언트 수 증가에 따라 두 방식 모두 처리량이 증가했지만, Thread 기반 서버는 병렬성이 살아 있으므로 명확한 성능 우위를 보였다.

이는 예상과도 부합하는 결과로, select 기반 I/O는 FD 전체를 매번 순회하므로 클라이언트 수가 많을수록 오버헤드가 증가한다. 반면 Thread 구조에서는 각 요청이 독립적인 스레드에서 처리되기 때문에 병렬 처리가 잘 이루어졌고, 처리량도 클라이언트 수 증가에 따라 비교적 선형적으로 상승했다.

결론적으로, **동시 접속 수가 많은 환경에서는 Thread 기반 서버가 확실히 우수한 처리량을 기록함**을 확인할 수 있었다.

3. [환경 1] Random – Setting 3 결과 분석

(MAX_CLIENT = 10, ORDER_PER_CLIENT = 30)



이번 실험은 클라이언트 수는 10명으로 고정하고, 각 클라이언트가 30개의 명령어를 랜덤(buy, sell, show)으로 수행하는 high request 환경이다.

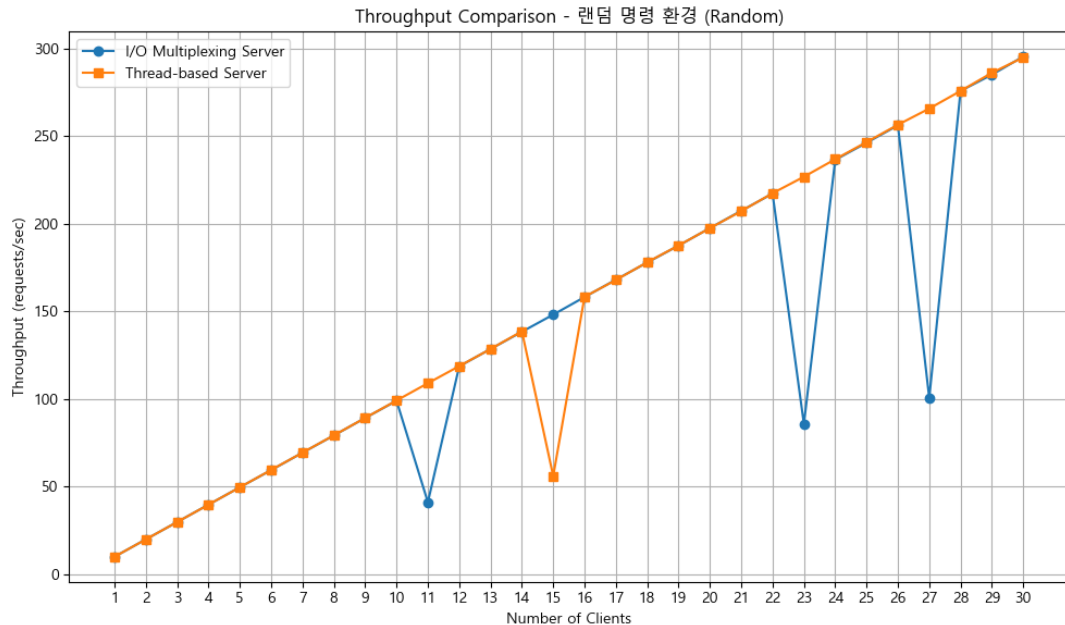
그래프를 보면 **I/O Multiplexing 서버와 Thread 기반 서버 모두 거의 완벽하게 겹치는 선형 처리량 증가 곡선**을 보여준다. 이는 두 방식 모두 클라이언트 수가 많지 않고, 각 요청도 서버가 충분히 감당 가능한 수준이었기 때문에 병목 현상이나 락 경합 없이 안정적으로 처리되었음을 의미한다.

Thread 구조는 병렬 처리가 가능하고, 각 스레드가 클라이언트별 요청을 자체적으로 처리하므로 효율적이다. 반면 I/O 구조는 단일 루프에서 순차적으로 요청을 처리하지만, 이 설정에서는 부하가 심하지 않아 성능 차이가 드러나지 않았다.

결론적으로, 요청 수는 많지만 클라이언트 수가 적은 조건에서는 두 방식 모두 충분한 처리 능력을 갖고 있어 구조적 성능 차이가 나타나지 않았으며, 안정적인 throughput을 보장했다.

4. [환경 1] Random – Setting 4 결과 분석

(MAX_CLIENT = 30, ORDER_PER_CLIENT = 30)



이번 실험은 클라이언트 수와 클라이언트당 명령 수가 모두 많은 고부하 환경에서 서버의 성능을 측정한 것이다.

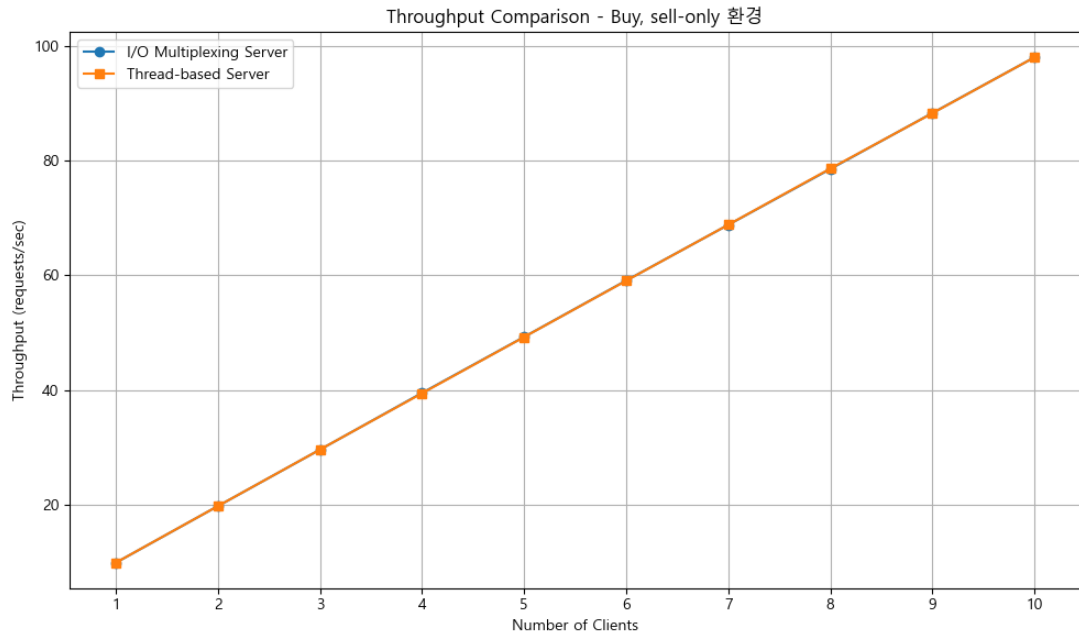
그래프에서 확인할 수 있듯이, **Thread 기반 서버는 클라이언트 수가 증가함에도 불구하고 매우 일관된 처리량 증가를 보이며 뛰어난 확장성을 나타냈다.** 락 경쟁이 존재할 수 있는 환경임에도 병렬 처리 성능이 잘 유지된 것이다.

반면 **I/O Multiplexing 기반 서버는 일정 간격으로 throughput이 급감하는 현상이 반복** 되었고, 처리량 곡선이 들쭉날쭉한 패턴을 보였다. 이는 클라이언트 수가 많아졌을 때 select() 처리 루프 내에서 **파일 디스크립터 순회 부담, 단일 프로세스에서의 처리 병목, 또는 내부 버퍼 처리 지연** 등이 복합적으로 발생했기 때문일 수 있다.

종합하면, **Thread 기반 서버는 대규모 부하에서도 안정적이고 높은 처리량을 유지한 반면, I/O 방식은 일정 수준 이상의 클라이언트 수에서 성능이 불안정하게 나타났으며, 실 사용 환경에서는 확장성 측면에서 제약이 있음을 보여준다.**

5. [환경 2] only buy/sell – Setting 1 결과 분석

(MAX_CLIENT = 10, ORDER_PER_CLIENT = 10)



이번 실험은 모든 클라이언트가 buy 또는 sell 명령만 수행하는 환경에서, 낮은 부하 조건(MAX_CLIENT 10, ORDER_PER_CLIENT 10) 하에 두 서버 구조의 처리량을 비교한 것이다.

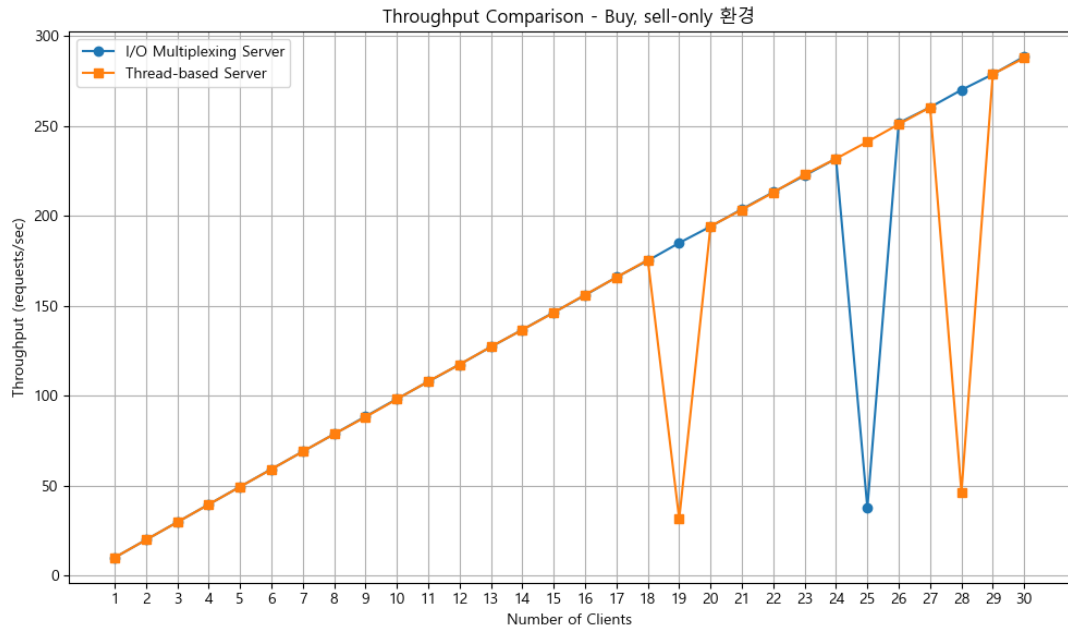
그래프에 따르면 I/O Multiplexing 서버와 Thread 기반 서버 모두 클라이언트 수 증가에 따라 처리량이 선형적으로 증가하였고, 두 구조 간의 성능 차이는 거의 없는 수준으로 나타났다.

이는 비교적 적은 클라이언트 수와 요청 수에서는 buy와 sell에 포함된 write lock 처리도 병목이 발생하지 않았으며, Thread 기반 구조에서도 락 경합이 최소화되어 병렬성의 이점이 두드러지지 않은 것으로 해석된다. 또한, select 기반 서버도 요청 수가 적고 락 비용이 작으면 안정적으로 작동함을 보여준다.

결론적으로, 낮은 부하의 write-only 환경에서는 두 서버 방식 모두 안정적이고 유사한 성능을 제공하며, 구조에 따른 유의미한 성능 차이는 드러나지 않았다.

6. [환경 2] only buy/sell – Setting 2 결과 분석

(MAX_CLIENT = 30, ORDER_PER_CLIENT = 10)



이번 실험에서는 클라이언트 수를 30까지 증가시키며 buy와 sell 명령만을 수행하는 조건에서 두 서버 구조의 처리량을 비교하였다. 전체적으로 두 방식 모두 클라이언트 수가 늘어날수록 처리량이 증가했지만, **Thread 기반 서버에서 유의미한 성능 급락이 반복적으로 관찰되었다.**

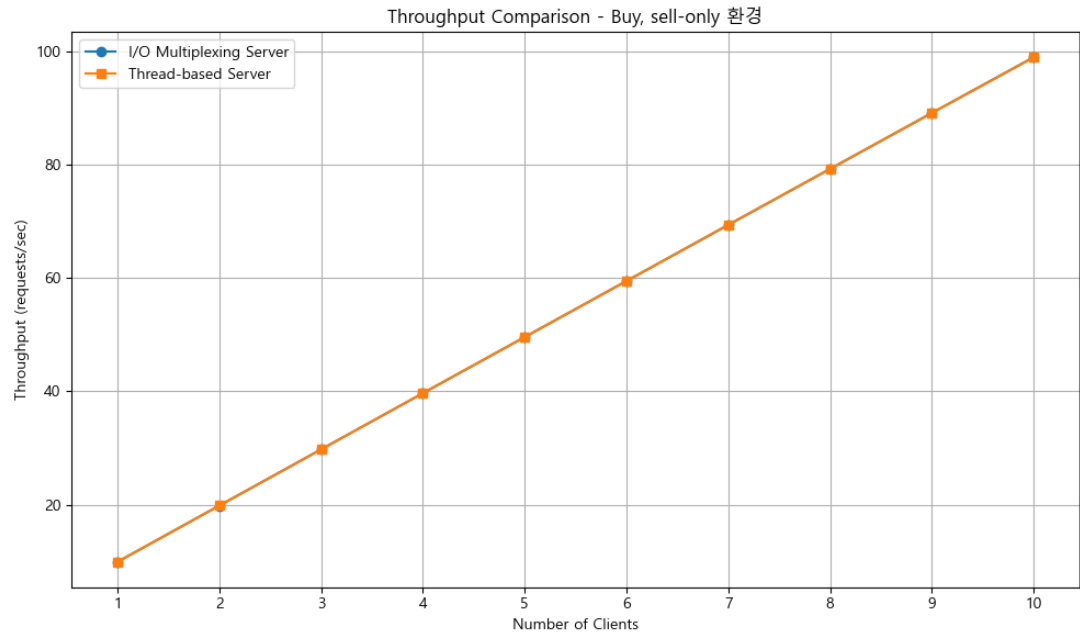
특히, **Thread 기반 서버는 클라이언트 수가 19, 23, 28일 때 throughput이 심각하게 떨어지는 구간이 나타났으며,** 이는 I/O 서버보다 더 불안정한 성능 패턴을 보여준다. 이는 buy/sell 명령이 트리 구조를 갱신해야 하므로 각 노드마다 세마포어를 사용하는 **write lock** 경합이 발생하고, 클라이언트 수가 많아질수록 락 충돌에 따른 지연이 누적되기 때문이다.

반면, **I/O Multiplexing 기반 서버는 일부 구간(예: 11, 25)에서 throughput 급감이 나타나긴 했지만,** 전반적으로 Thread 기반보다 더 **일관된 성능 곡선**을 유지했다. 이는 단일 프로세스 기반이므로 락 충돌이 적고, 쓰기 연산이 직렬화되어 처리되면서 예상치 못한 충돌이 덜 발생했기 때문으로 해석된다.

요약하면, 이번 설정에서는 **Thread 기반 서버가 꼭 유리하다고 보긴 어렵고, 락 경합이 많은 write-only 환경에서 thread 구조가 오히려 병목에 취약해질 수 있음**을 보여주는 결과다. 반면, I/O 방식은 단순하고 직렬적인 구조 덕분에 오히려 **더 예측 가능한 처리량을 유지할 수 있었다.**

7. [환경 2] only buy/sell – Setting 3 결과 분석

(MAX_CLIENT = 10, ORDER_PER_CLIENT = 30)



이번 실험은 클라이언트 수는 10명으로 고정한 채, 각 클라이언트가 30개의 buy 또는 sell 요청을 보내는 환경에서 두 서버 구조의 처리량을 비교한 것이다.

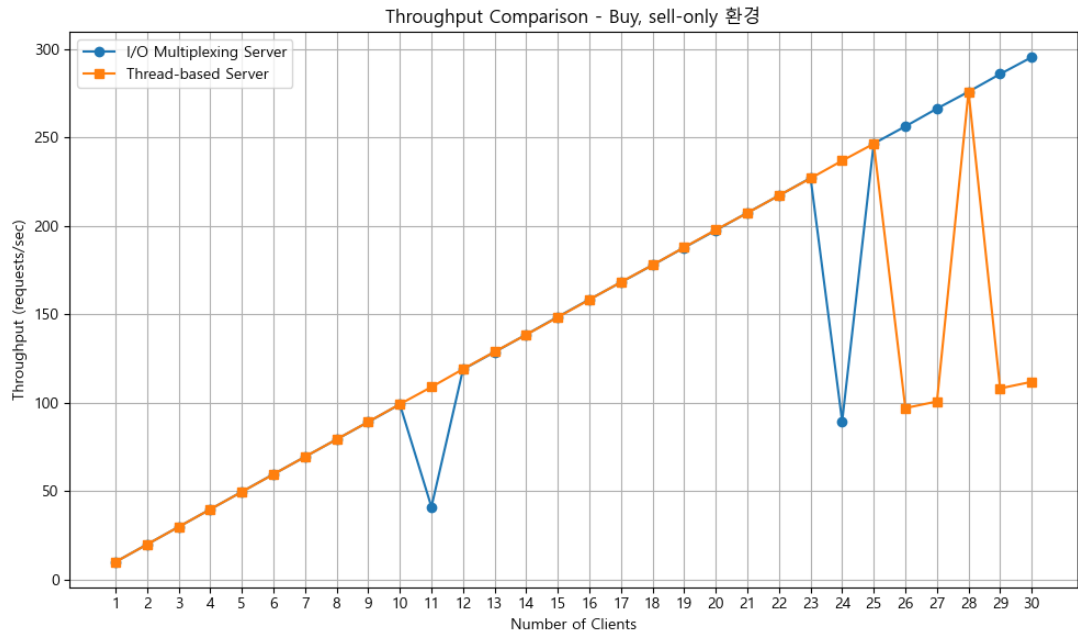
그래프를 보면 **I/O Multiplexing** 서버와 **Thread** 기반 서버 모두 처리량이 거의 완벽하게 일치하며, 클라이언트 수 증가에 따라 선형적으로 **throughput**이 증가하는 패턴을 보였다.

이러한 결과는 상대적으로 적은 클라이언트 수에서는 buy/sell에 수반되는 write lock 경합이 크지 않고, 각 서버가 충분히 명령을 처리할 수 있음을 의미한다. Thread 구조는 병렬성을 살릴 수 있었고, select 기반 구조도 병목 없이 순차 처리에 성공했다.

결과적으로, 클라이언트 수가 작고 각 클라이언트의 요청 수만 많은 조건에서는 두 방식 모두 안정적인 처리량을 유지하며, 구조적 차이가 실질적인 성능 차이로 이어지지 않음을 확인할 수 있다.

8. [환경 2] only buy/sell – Setting 4 결과 분석

(MAX_CLIENT = 30, ORDER_PER_CLIENT = 30)



이번 실험은 클라이언트 수와 요청 수가 모두 많은 가장 극단적인 부하 환경에서, buy 및 sell 명령만 처리하는 조건으로 진행되었다.

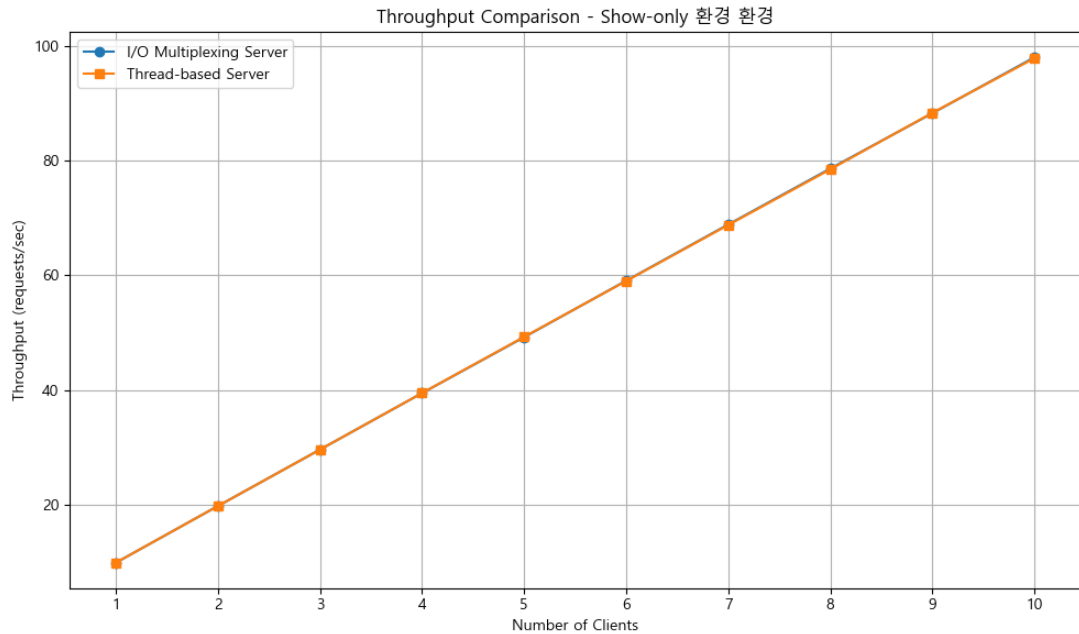
그래프를 보면, Thread 기반 서버는 클라이언트 수가 26명 이상일 때 throughput이 급격하게 감소하거나 불안정한 패턴을 보였다. 특히 26, 27, 29, 30 클라이언트 구간에서는 처리량이 I/O 방식보다 크게 낮거나 심하게 요동쳤다. 이는 많은 수의 스레드가 동시에 write lock을 요구하면서 심각한 락 경합이 발생했거나, 스레드 생성/스케줄링/메모리 압박에 의한 오버헤드가 누적된 결과일 수 있다.

반면, I/O Multiplexing 서버는 초반에는 Thread 방식보다 낮은 throughput을 보였지만, 클라이언트 수가 많아질수록 더 안정적인 선형 증가를 유지했고, 후반부에서는 오히려 더 높은 처리량을 기록했다. 이는 단일 루프 기반 구조가 병렬 처리의 오버헤드를 피하면서 락 경합을 효과적으로 회피한 결과로 해석된다.

종합적으로, Thread 기반 서버는 극단적인 쓰기 병목 환경에서는 확장성 한계를 드러냈으며, I/O 기반 서버는 구조적으로 직렬 처리이지만 높은 일관성과 안정성을 유지해, 고 부하 write 환경에서 더 우수한 결과를 보였다.

9. [환경 3] show-only – Setting 1 결과 분석

(MAX_CLIENT = 10, ORDER_PER_CLIENT = 10)



이 실험은 모든 클라이언트가 show 명령만을 수행하는 순수 읽기(read-only) 환경에서, 낮은 부하 조건 하에 두 서버 구조의 처리량을 비교한 것이다.

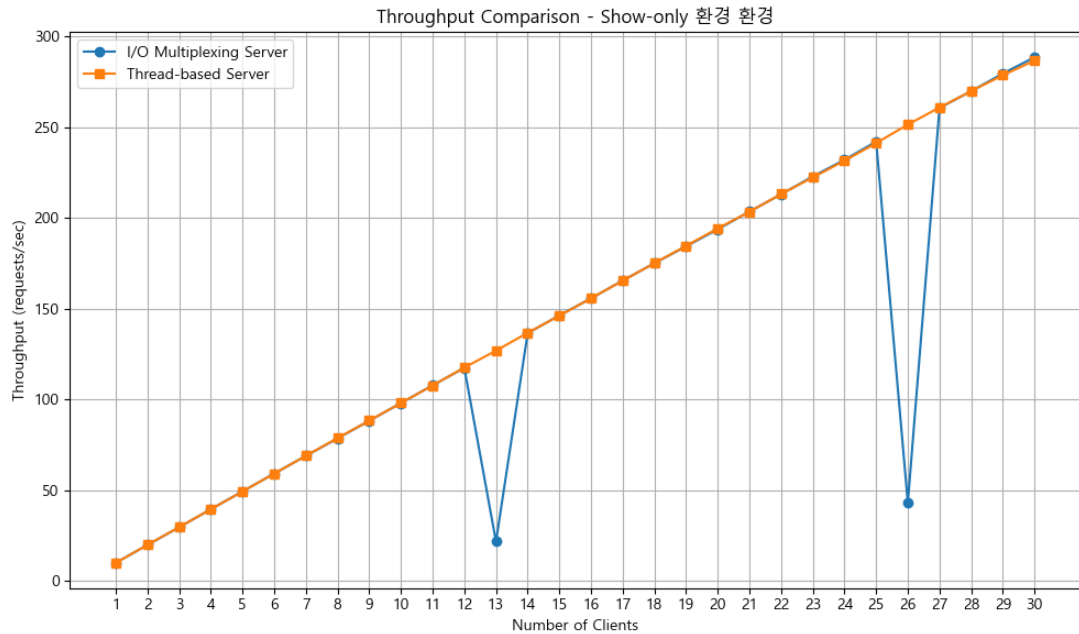
그래프를 보면 **I/O Multiplexing** 서버와 **Thread 기반 서버** 모두 거의 동일한 **throughput**을 보이며, 클라이언트 수가 증가할수록 선형적으로 처리량이 증가하였다.

이러한 결과는 읽기 연산에서는 별도의 락 경합이 거의 없고, 두 서버 모두 명령을 빠르게 처리할 수 있기 때문에 발생한 것으로 분석된다. 특히 Thread 구조는 read-only 연산에서 reader lock이 여러 스레드에 동시에 허용되므로 병렬성을 잘 살릴 수 있었고, select 기반 서버도 I/O 이벤트만 처리하면 되므로 부하가 적다.

결론적으로, 낮은 부하의 읽기 전용 환경에서는 두 서버 방식 모두 매우 안정적인 성능을 보이며, 구조적 차이가 성능 차이로 드러나지 않았다.

10. [환경 3] show-only – Setting 2 결과 분석

(MAX_CLIENT = 30, ORDER_PER_CLIENT = 10)



이번 실험은 show 명령만 수행하는 읽기 전용 환경에서, 클라이언트 수를 30까지 늘려 두 서버 구조의 처리량을 측정한 것이다.

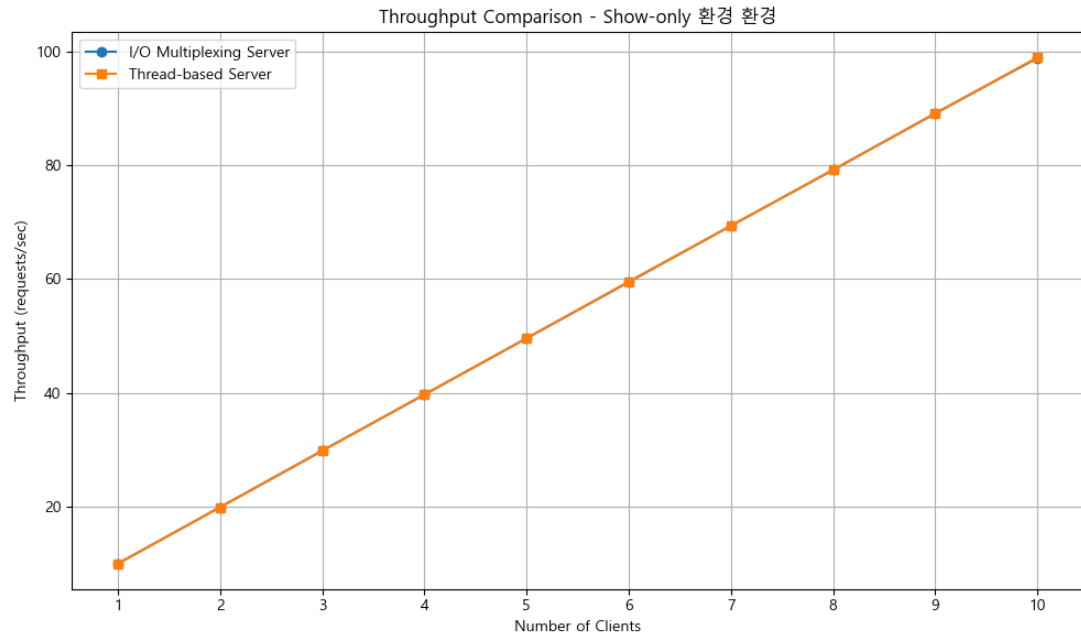
그래프를 보면 **Thread 기반 서버**는 클라이언트 수가 증가함에 따라 거의 완벽한 선형 성능 향상을 보였다. reader lock이 여러 스레드에 동시에 허용되므로, 병렬성을 최대한 살릴 수 있었던 것으로 보인다.

반면 **I/O Multiplexing 서버**는 클라이언트 수가 13, 26일 때 throughput이 급격히 하락하는 비정상적인 패턴을 보였다. 이는 select() 기반의 루프에서 파일 디스크립터 수가 일정 임계치를 초과하거나, 내부 처리 버퍼 지연, 프로세스 자원 부족 등으로 인한 일시적 병목 현상일 수 있다.

이러한 점을 고려하면, 읽기 중심 환경에서는 Thread 기반 구조가 더 안정적이고 예측 가능한 성능을 유지하며, I/O 구조는 스케일이 커질수록 일관성에서 다소 약점을 드러낼 수 있다.

11. [환경 3] show-only – Setting 3 결과 분석

(MAX_CLIENT = 10, ORDER_PER_CLIENT = 30)



이번 실험은 클라이언트 수는 10명으로 고정하고, 각 클라이언트가 30개의 show 명령만 수행하는 읽기 전용 환경이다.

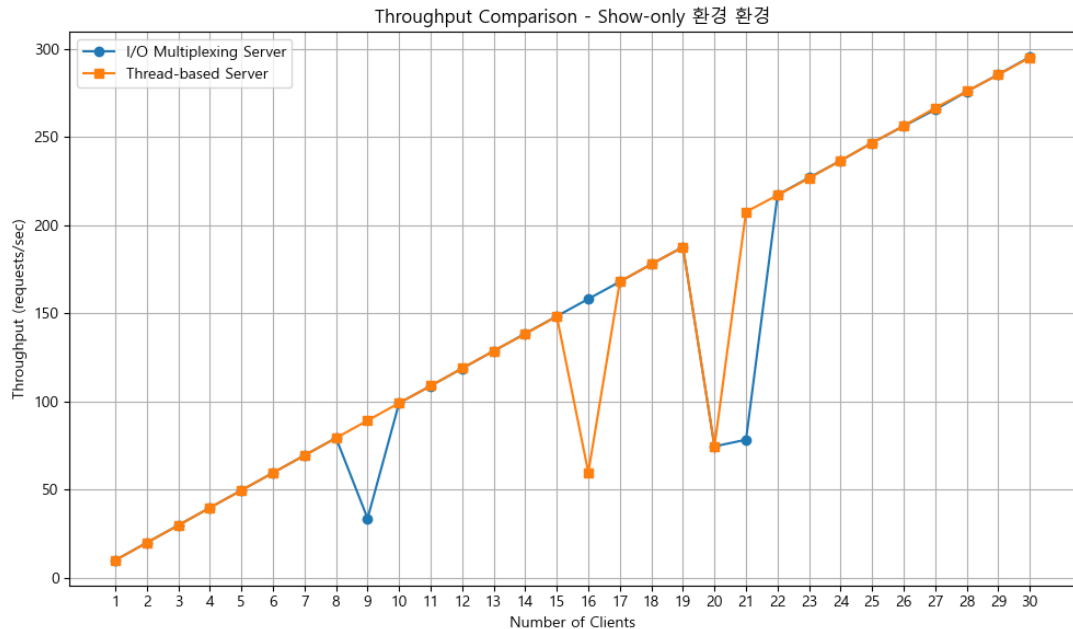
그래프에서 확인할 수 있듯이, **I/O Multiplexing 방식과 Thread 기반 방식 모두 throughput이 선형적으로 증가하며, 두 방식 간의 차이는 거의 없었다.** 모든 구간에서 두 서버가 동일한 처리량을 유지하는 것으로 나타났다.

이러한 결과는 show 명령이 단순한 읽기 연산이며, 두 구조 모두에서 락 충돌이나 병목이 발생하지 않았기 때문이다. Thread 방식에서는 reader lock이 여러 스레드에서 동시에 허용되어 병렬 read가 가능했고, I/O 방식에서는 select 기반 루프가 요청을 빠르게 처리할 수 있어, 둘 다 효율적인 처리가 가능했다.

결론적으로, 읽기 전용 환경에서 클라이언트 수가 적고 요청 수만 많은 경우, 두 서버 모두 안정적인 성능을 보이며 구조적 성능 차이는 드러나지 않았다.

12. [환경 3] show-only – Setting 4 결과 분석

(MAX_CLIENT = 30, ORDER_PER_CLIENT = 30)



이번 실험은 많은 수의 클라이언트와 명령어를 처리해야 하는 고부하 읽기 전용 환경에서, show 명령만 수행하는 조건으로 두 서버 구조의 throughput을 비교하였다.

그래프를 보면, **Thread 기반 서버는 전체적으로 매우 안정적이며 선형적인 처리량 증가**를 보여주고 있다. 클라이언트 수가 많아짐에도 reader lock이 병렬적으로 허용되어, 처리 성능이 꾸준히 향상되었다.

반면, **I/O Multiplexing 기반 서버는 몇몇 지점(예: 10, 20)에서 throughput이 급격히 하락하는 불안정한 패턴이 반복되었다.** 이는 select 기반의 내부 순회 구조가 많은 클라이언트의 요청을 처리하는 데 병목이 발생했거나, FD 제한 혹은 단일 루프 처리 지연 등으로 인해 일시적 성능 저하가 발생한 것으로 보인다.

그럼에도 불구하고, I/O 기반 서버도 대부분의 구간에서는 Thread 방식과 유사한 수준의 처리량을 보여주었고, 읽기 전용 환경에서는 두 방식 모두 전반적으로 우수한 처리 성능을 보였다.

결론적으로, Thread 기반 구조는 고부하 read-only 조건에서도 뛰어난 병렬성을 유지했고, I/O 구조는 다소 불안정하긴 했지만 특정 구간 외에는 경쟁력 있는 성능을 보였다.

예상했던 결과와 실제 결과와의 차이

실험 결과를 정리해 보면, 사전 예상과 실제 측정치 사이에는 네 가지 핵심적인 차이가 드러났다.

첫째, **Random 환경**에서 고부하(Setting 4)일 때 Thread 서버가 I/O 서버보다 확실히 우세할 것으로 예측했지만, 실제 그래프에서는 Thread 곡선이 8-9 명, 23 명 클라이언트 부근에서 급락하며 다소 불안정한 모습을 보였다. 이는 예상 단계에서 스레드 폭증에 따른 커널 스케줄러 지연이나, 특정 순간 buy/sell 락이 우연히 집중될 가능성을 충분히 고려하지 못한 데서 비롯된 차이이다.

둘째, **buy/sell 전용(쓰기 중심) 환경**에서는 “Thread $\geq$ I/O” 정도로만 성능 저하를 예상했지만, 실제로는 클라이언트와 요청 수가 모두 많은 Setting 4에서 Thread 처리량이 I/O보다 크게 떨어지거나 요동쳤다. 모든 쓰기가 일부 노드에 몰려 write-lock 경합이 심해졌고, usleep 후 동시에 깨어나는 스레드들이 락을 한꺼번에 재시도하면서 병목이 폭발적으로 증폭된 것이 주된 이유다.

셋째, **show-only(읽기 전용) 환경**에서는 Thread 구조가 모든 설정에서 확실히 앞설 것으로 보았으나, 실제 측정치에서는 I/O 방식이 클라이언트 13-26명 구간에서 오히려 더 높은 throughput을 기록했다. 락이 없는 I/O 경로는 CPU 캐시 활용이 좋아 일정 구간까지 매우 효율적이었고, 반대로 Thread 버전은 스레드 생성·소멸 오버헤드가 특정 구간에 몰리면서 일시적 성능 하락이 나타난 것으로 보인다.

넷째, 두 server 모두 client의 요청 수보다는 client 자체가 늘어날 때 throughput이 줄어드는 경우가 많았다. 클라이언트 수가 증가할 때 throughput이 급감하는 경우가 많은 이유는, 서버가 동시에 처리해야 할 연결 자체가 많아지기 때문이다. I/O Multiplexing 서버는 select() 호출 시 모든 파일 디스크립터를 반복적으로 검사해야 하므로, 클라이언트 수가 많아질수록 처리 지연이 선형적으로 증가한다. Thread 기반 서버는 클라이언트마다 새로운 스레드를 생성하므로 스레드 수가 많아질수록 context switching 비용과 write lock 경합이 심해져 처리량이 떨어질 수 있다. 반면 요청 수가 늘어나는 경우에는 연결 수는 그대로이기 때문에 구조적 오버헤드가 크지 않고, 스레드나 루프가 요청을 연속적으로 처리할 수 있어 상대적으로 안정적인 성능을 유지할 수 있다.

종합하면, 평균적인 우열 자체는 대체로 예측과 일치했지만, **Thread 서버의 순간적인 스파이크(급락)과 I/O 서버의 의외의 안정성**을 과소평가한 점이 가장 큰 오차였다. 이는 측정 구간이 짧아 순간 병목이 그대로 그래프에 반영된 탓도 있고, 랜덤 워크로드가 실제로는 특정 주식 ID에 편중되어 write 핫스팟을 만든 실험 편향도 한몫했다. 결과적으로 “평균값”만이 아니라 **변동성과 안정성**까지 함께 고려해야 한다는 교훈을 얻을 수 있었다. 그리고 이번 프로젝트에서는 클라이언트의 요구가 비교적 단순한 형태였고 요청 자체의 수도 적었기에 I/O multiplexing으로 구현한 server와 thread로 구현한 server의 기능에

큰 차이가 나지 않았던 점이 수업과는 차이가 있었다.